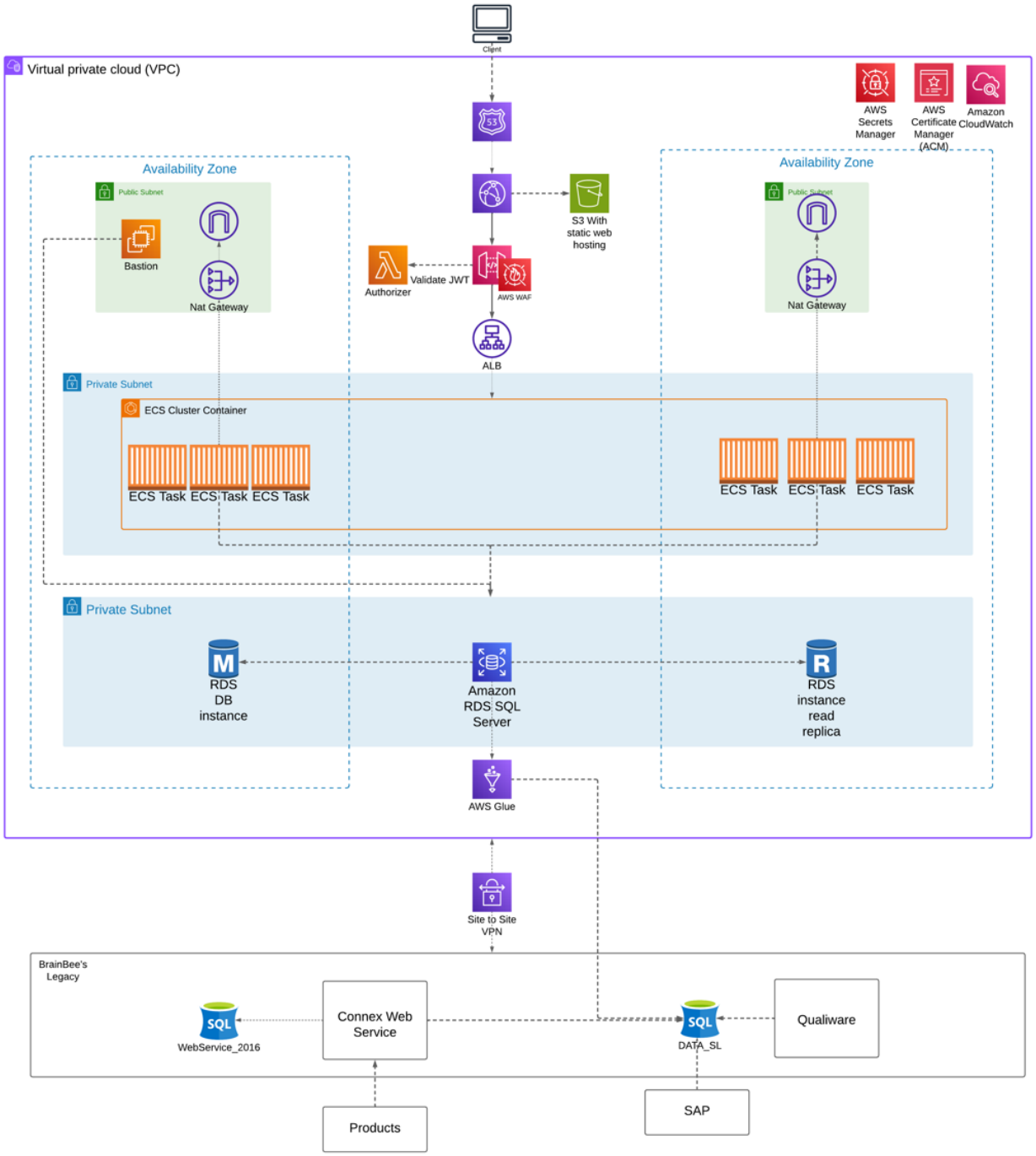
Architecture



API Gateway

- **Request Routing**

    - The API Gateway will route incoming requests to the appropriate ECS task based on the request parameters.

- **Authorization**
    - It will invoke a token authorizer to:
        - Validate the token.
        - Extract the user's role from the token.
        - Check whether the user is authorized to perform the requested action.
- **Tracing**
    - A unique `traceId` will be generated by the API Gateway.
    - This ID will be added to the `X-TraceId` header and forwarded with the request to the ECS task.
    - The ECS task will include this `traceId` in all logs to facilitate request-specific log analysis.
- **API Exposure**
    - The API Gateway will expose RESTful APIs over HTTPS to ensure secure communication.
- **Rate Limiting**
    - Rate limiting will be configured to prevent abuse and ensure fair usage of the service.

Database

High Availability Configuration

- Amazon RDS will be configured with **Multi-AZ** and **one standby instance**.
- In this setup:
    - A **primary database instance** is created.
    - Data is **synchronously replicated** to a **standby instance** located in a different Availability Zone (AZ).
    - This forms a **classic active-passive architecture**.

- **Failover Mechanism**
    - The **standby instance** is used **exclusively for failover**.
    - In case of a primary instance failure:
        - Amazon RDS **automatically fails over** to the standby.
        - The **DNS record** of the database endpoint is updated to point to the standby, which becomes the new primary.
        - This process is **automated**, requiring **no manual intervention**, and helps **minimize downtime**.

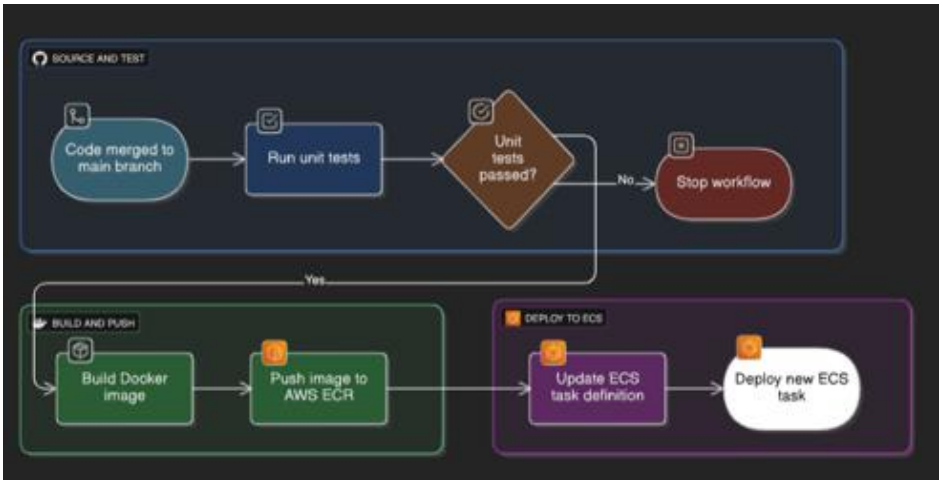
Logging and monitoring

We will use cloud watch to log all API requests and all business transactions in the ECS tasks which can help in debugging.

We could have dashboard to track system health and metrics which can help us in understanding the system health.

CI/CD

We will have a CI/CD using github action, it will basically run unit test, build docker image then push it to ECR, then create a new ECS task definition and deploy a new ECS task



Compute

We will have 2 Microservices, one for user-management and the other one will be for the device-management. Each one will have its own docker image and its ECS task definition and ECS task.

In future we might have another service for notification, but for now notifications is only sent from user-management service

All ECS tasks will be running in a private subnet, which means they will be only reachable through the API gateway.

Auto Scaling

We will be scaling in and out by using ASG (Auto Scaling Group), we will have rules based on CPU utilization, Memory utilization and request processing time.

Security

1. Web Application Firewall (WAF)

- **WAF** will be deployed on the **API Gateway** to protect against common web exploits and malicious traffic.

2. Secure ETL Communication

- Data exchange between the **cloud database** and the **on-premises database** (ETL process) will be secured using a **site-to-site VPN** connection.

3. Encryption in Transit

- All HTTP requests will use **TLS (HTTPS)** to ensure secure communication across services.

4. Authentication & Authorization

- All incoming requests will include a **JWT token**.
- The token will be validated by a **Lambda Authorizer** integrated with the **API Gateway**, enforcing access control.

5. RDS Security Group Configuration

- The **RDS security group** will include an **inbound rule** allowing traffic from the **ECS security group** on **port 1433** (SQL Server default port).

6. Bastion Host (Jump Box) Access

- A **Bastion Host** may be used to allow **administrators** to securely connect to the RDS instance.
- Access will be established via **SSH tunneling**.
- The **RDS security group** will include an **inbound rule** to permit connections from the Bastion Host.

Secrets Management & Encryption

Secrets Management with AWS Secrets Manager

- **AWS Secrets Manager** will be used to securely store all sensitive credentials, including:
  - Database usernames and passwords
  - API keys
- **ECS Integration**:
  - ECS tasks will retrieve secrets directly from Secrets Manager.
  - This is configured by referencing the secret in the **ECS task definition**, ensuring secure and dynamic access without hardcoding credentials.

Encryption in Transit with ACM & CloudFront

- **AWS Certificate Manager (ACM)** will be used to issue **TLS certificates**.
- These certificates will enable **encryption in transit** for all communications.
- **TLS Termination** will occur at **Amazon CloudFront**, ensuring secure delivery of content to end users while offloading the encryption overhead from backend services.

Networking

We will be having 2 AZs

Each **Availability Zone (AZ)** includes the following components:

- **1 Public Subnet**:
  - Connected to an **Internet Gateway** for inbound/outbound internet access.
  - Contains a **NAT Gateway** to enable secure outbound internet access for private subnets.
- **2 Private Subnets**:

  1. **ECS Cluster Subnet**:
     - Hosts the **ECS Cluster**.
     - Accessible via the **Application Load Balancer (ALB)**.
     - Can initiate outbound connections to the internet through the **NAT Gateway**.
  2. **Database Subnet**:
     - Hosts the **Database Cluster**.
     - Isolated from direct internet access for enhanced security

Cost Drivers

We have **two main components** that significantly affect cost:

1. **Compute**
2. **Database**

Compute Options

We can choose between **ECS with Fargate** or **ECS with EC2**.

ECS with Fargate:

- Easier to deploy
- Simplifies scaling in and out
- Has default integration with CloudWatch

- More expensive for long-running tasks

ECS with EC2:

- More cost-effective for long-running workloads
- More complex to deploy
- Scaling in and out requires more effort
- Requires manual integration with CloudWatch

Database Options

RDS:

- AWS handles backups, patching, failover, and maintenance.
- Built-in support for Multi-AZ deployments and automated failover.
- Easy to scale vertically (instance size) and storage.
- Integrated with IAM, VPC, and encryption options.
- Native integration with CloudWatch and performance insights.
- Automated backups and point-in-time recovery.
- Limited Customization and Limited access to the underlying OS and database engine configurations.
- Can be more expensive than self-managed solutions, especially at scale.

Self Managed Database:

- Complete access to OS, database engine, and configurations.
- Can install any extensions, plugins, or custom scripts.
- Cost Less than RDS, especially with Reserved EC2 instances.
- We must manage backups, patching, monitoring, and failover.
- Requires more expertise to set up and maintain securely and reliably.
- Manual effort needed to scale and ensure high availability cluster.

MAHLE internal restricted (CL2)